

Looking at the current code, this is already half-built: fusion/engine.py does confidence-aware fusion, but it's hardcoded to exactly two sources (macro, perception). forecast/models.py has two forecasting baselines (HistoricalAverageForecaster, SpatialTemporalForecaster) that don't yet feed into fusion at all. To get to "4 models, brain decides on whatever subset is alive," you need one shared output contract and a fusion function generalized from 2 inputs to N.

1. Common output contract (every forecaster emits this, nothing else)

Reuse/extend schema.RoadObservation — it already has exactly the fields a fusion/decision layer needs:

```
class RoadObservation(StrictModel):  
    road_id: str  
  
    source: SourceKind    # which model produced this  
  
    travel_time_s: float  # the actual prediction  
  
    stddev_s: float      # uncertainty — this is the key field  
  
    confidence: float     # 0-1  
  
    age_s: float = 0  
  
    distance_m: float | None = None  
  
    blockage: bool = False
```

Add two more SourceKind members: HISTORICAL and SPATIAL_TEMPORAL, alongside existing MACRO and PERCEPTION. That's your 4 models. Each one wraps its own internal output (SpeedForecast, a Google Routes duration, a perception cluster estimate, whatever) and converts it into a RoadObservation before handing it to the brain. The brain never sees model-specific internals — only this contract.

2. Why stddev_s is what makes "decide from 1, tolerate any failing" work

The current fuse_observations already does inverse-variance weighting: $\text{weight} = 1/\text{stddev}^2$, and a missing observation just isn't in the list — it doesn't special-case "2 sources," it just currently only accepts 2 parameters. Generalize the signature to `list[RoadObservation]`:

```
def fuse_observations(road_id: str, observations: list[RoadObservation], config:
FusionConfig | None = None) -> RoadEstimate:
```

```
    observations = [o for o in observations if o is not None]
```

```
    if not observations:
```

```
        raise ValueError("at least one observation is required")
```

```
    # blockage override check stays the same, scanning all observations
```

```
    # inverse-variance weighting loop already generalizes to N with no changes
```

With that change: 4 models alive → weighted blend of 4. 1 model crashes → its RoadObservation is just absent from the list, no code path changes, confidence naturally drops because precision_sum drops. 1 model alive → fusion degenerates to that model's own estimate (this already happens today with `len(adjusted) == 1`). Zero models alive → the one explicit failure case, raise/skip that road for routing.

3. Model-specific responsibility: never let a bad model crash the pipeline

Each of the 4 forecasters should be called through a wrapper that turns exceptions into "absent observation," not a propagated error:

```
def safe_predict(forecaster, road_id, ...) -> RoadObservation | None:
```

```
    try:
```

```
        return forecaster.predict(...)
```

```
    except Exception:
```

```
        return None
```

That's the actual mechanism for "brain works even if one of the four fails" — failure becomes None in the list, not an exception the brain has to catch.

4. Confidence output, not just travel time

The brain (routing/GOAT) shouldn't consume raw travel times — it should consume RoadEstimate.confidence and .blockage too, exactly as routing/ presumably does today via RoadEstimate. That's why the existing blockage-override short-circuit in fuse_observations matters: a single high-confidence blockage report from one model should override a rosy average from the other three, which is a decision-quality requirement, not just a data-fusion one.

Want me to implement the generalization — add HISTORICAL/SPATIAL_TEMPORAL to SourceKind, rewrite fuse_observations to take list[RoadObservation], and wire forecast/models.py's two forecasters to emit RoadObservation so all 4 sources actually flow into fusion/engine.py?